access to computing resources. Static authorization has several significant limitations that dynamic authorization systems overcome.

1.2.1 Scalability

As the number of principals, actions, and resources increases, managing static lists becomes increasingly difficult. (We will use the word *principal* rather than user to refer to the entities that are taking an action since many of these entities are machines or processes rather than humans.) Each addition requires that lists be updated, either manually or through some rule. Typically, there is a list for each resource or group of resources. Putting thousands, even millions, of customers and employees on multiple lists is unwieldy.

For example, I once worked at a large organization that used groups as its primary means of controlling access. Over the years, the number of groups had expanded until there were tens of thousands. The organization had various workarounds in place to automatically keep the groups up to date, but administrators frequently had to make manual updates to add or delete access to something or for someone.

1.2.2 Flexibility

Static lists can’t easily adapt to changing request context, organizational structure, or compliance requirements. Request context is a particularly tricky problem. Request context includes things like the time of the request, the location of the requester, whether she’s on a company-managed device, and so on.

Should Alice from Finance have after-hours access to transfer funds? Static lists don’t know. Solving the problem requires building special-purpose code in the finance system to augment the static authorization system.

Alternatively, consider scenarios involving elevated or sensitive access—commonly managed through Privileged Access Management (PAM). Access to production systems, for example, might be granted only to developers who are currently on call, based on just-in-time information. In some cases,